Applied AI Engineer — Case Study Assessment

SMB Catalyst × IIT Bombay Campus Recruitment 2026

⏱ **Duration: 48 Hours** | 📦 **Deliverables: Working Demo + PRD + Presentation**

Overview

You are given **48 hours** to solve one of the two case studies below. This is not a theoretical exercise - we want to see a **working demo** that proves you can take an ambiguous real-world problem, think through it systematically, and ship something tangible using modern AI-enabled development tools.

What we're really evaluating: Your ability to think clearly under ambiguity, use AI tools as a multiplier (not a crutch), make smart trade-offs given a tight deadline, and deliver something that actually works - not just a slide pitch.

What You Must Deliver

1. Product Requirements Document (PRD)

A concise document (2–4 pages) covering: problem understanding, proposed approach, architecture decisions, trade-offs considered, and what you chose to build vs. descope given the 48-hour constraint. Show us your thought process, not just the output.

2. Working Demo / Prototype

A functional system that takes the specified input and produces meaningful output. It doesn't need to be production-grade — but it must work end-to-end. A demo that processes one real example convincingly beats a slide deck claiming to solve everything.

3. Presentation (10–15 minutes)

Walk us through: problem understanding → approach → architecture → demo → what you'd build next with more time. Be prepared for deep technical questions.

AI-Enabled Development — Expected Approach

We expect you to leverage AI tools throughout your development process. This is not optional — it's a core part of what we're evaluating. **Show us how you use AI as a force multiplier.**

Tools you should consider using:

- **Coding Assistants:** Claude Code, GitHub Copilot, Cursor, Windsurf — for rapid prototyping, debugging, and code generation.
- **Research & Architecture:** Use LLMs to explore the problem space, evaluate approaches, and stress-test your architecture decisions.
- **Domain Knowledge:** Use AI to quickly ramp up on jewelry CAD / architectural drafting specifics you may not know.
- **Testing & Iteration:** Use AI to generate test cases, validate outputs, and iterate quickly on your prototype.

Important: Document your AI usage. In your PRD, include a brief section on which AI tools you used, how they accelerated your work, and where they fell short. We value honest reflection over performative claims.

Case A: 2D → 3D Jewelry Design Automation

Context

High-end jewelry brands follow this workflow:

- Concept → 2D design (hand sketch / CorelDRAW) with dimensions, stone placements, and budget constraints.
- CAD designers manually build 3D models in Rhino or Matrix (jewelry-specific Rhino plugin).
- Final 3D output is used for 3D printing (wax/resin) → lost-wax casting → finished piece.

Current pain points: 1–3 days per design, heavy reliance on skilled CAD talent, and limited scalability for custom orders.

Problem Statement

Design and build a system that automates or substantially reduces manual CAD effort in converting 2D jewelry designs into production-ready 3D CAD models.

Expected Input / Output

- **Input:** 2D sketch / vector file with annotations (dimensions, stone types, metal spec).
- **Output:** Parametric 3D model (editable geometry, stone settings, prongs/mounts) → exportable as STL / 3DM compatible with Rhino/Matrix.

Scope — Address These Elements

- **Input Processing:** Sketch/vector → structured geometry extraction (detect curves, symmetry axes, stone positions).
- **Geometry Reconstruction:** Curves, surfaces, symmetry, thickness inference from 2D to 3D.
- **Feature Modeling:** Stone placement, prongs, joints, pavé logic, mount types.
- **Constraint Handling:** Precise dimensions (mm-level), metal weight estimation, manufacturability (casting rules, wall thickness, durability).
- **Output Generation:** Fully editable CAD model with a validation step before export.
- **End-to-End Pipeline:** Input → processing → model → validation → export.

Key Considerations

- Millimeter-level precision is required — jewelry is manufactured to tight tolerances.
- Parametric vs. generative modeling trade-offs (editability vs. speed).
- Handling noisy/incomplete 2D inputs (hand sketches vs. clean vectors).
- Seamless integration with existing Rhino/Matrix workflows (file formats, layer conventions).
- Manufacturability constraints for real-world lost-wax casting.

Case B: Automated Layout & Drafting System (AutoCAD/BIM)

Context

Typical architecture / interior design workflow:

- Client brief (often unstructured text/notes).
- Architect develops high-level concepts.
- Draftsman manually creates AutoCAD drawings (floor plans, sections, elevations).
- Iterations follow before moving to visualization (SketchUp) or full BIM (Revit).

Current pain points: 6–12 hours per layout iteration, repetitive rule-based drafting, heavy dependence on draftsmen, and frequent rework due to interpretation gaps.

Problem Statement

Design and build a system that generates first-pass architectural layouts and AutoCAD-ready drafting outputs directly from client briefs, materially reducing or eliminating manual drafting effort.

Expected Input / Output

- **Input:** Unstructured brief (e.g., "3BHK, 1500 sq ft, open kitchen, 1 study, maximize natural light").
- **Output:** Zoned floor plan with room dimensions, spatial allocation, door/window placement, basic furniture layout → DWG/DXF file (editable, not image-based).

Scope — Address These Elements

- **Input Processing:** Unstructured brief → structured requirements (rooms, areas, constraints, adjacencies).
- **Layout Engine:** Zoning (public/private/service), adjacency & circulation logic, space allocation under area constraints.
- **Drafting Engine:** Walls, openings, annotations, auto-dimensions, standardized CAD layers.
- **Output Generation:** AutoCAD / Revit-compatible files (DWG/DXF).
- **End-to-End Pipeline:** Input → layout → drafting → iteration loop.

Key Considerations

- First-pass usability vs. final construction accuracy (optimize for a useful starting point, not perfection).
- Rule-based vs. ML/hybrid approach — justify your choice.
- Constraint enforcement: ergonomics, adjacency rules, basic building code logic.
- Handling edge cases: irregular plots, conflicting requirements, multi-story.
- Editable output with a human override layer (architect must be able to modify).

Evaluation Criteria

You will be scored across 5 dimensions. Each is weighted to reflect what matters most for the Applied AI Engineer role at SMB Catalyst.

Dimension	Weight	What We're Looking For	Red Flags
Working Demo & Prototype	**30%**	End-to-end functional prototype that takes real input and produces meaningful output. Handles at least one realistic example convincingly. Clean, modular, well-structured code. Shows engineering craft, not just hacking.	Slide deck with no working code. Demo that only works on hardcoded inputs. Spaghetti code. No error handling.
Thought Process & PRD	**25%**	Clear problem decomposition. Smart scoping — what to build vs. skip in 48 hrs. Architecture that reflects real constraints. Trade-off analysis showing engineering maturity. Honest about limitations.	Generic copy-pasted architecture. No trade-off discussion. PRD reads like raw AI output with no original thinking.
AI-Enabled Development	**20%**	Evidence of using AI tools effectively — not just for code gen but for research, architecture, testing. Honest documentation of what worked and didn't. Shows you can direct AI, not just copy its output.	No AI usage documented. Blind copy-paste without understanding. Claims AI wrote everything with no ownership.
Technical Depth & Domain Understanding	**15%**	Understanding of domain constraints (jewelry tolerances / drafting standards). Smart tech stack choice with justification. Awareness of edge cases and failure modes. Validation approach.	Ignores domain constraints. No error handling. Chose tools without justifying why. No validation.
Presentation & Communication	**10%**	Clear narrative: problem → approach → demo → learnings. Concise and confident. Anticipates questions. Honest about limitations. Shows genuine curiosity about the problem domain.	Reading from slides. Can't explain own architecture. Defensive about shortcomings. No demo during presentation.
TOTAL	**100%**		

Scoring Tiers

Tier	Score	What It Means
Outstanding	85–100	Exceptional problem-solving, polished working demo, strong AI usage, clear communication. Immediate hire signal.
Strong	70–84	Solid demo and thinking, effective AI usage, minor gaps. Strong hire signal.
Competent	55–69	Functional demo but shallow depth, or strong thinking with weak execution. Further evaluation needed.
Below Expectations	< 55	Incomplete demo, poor scoping, no AI development evidence, or superficial understanding.

What SMB Catalyst Values

We build AI solutions for real businesses — not research papers. Here's what that means for this assessment:

- **Bias for action over perfection.** Ship something that works in 48 hours. A rough but functional demo beats a perfect slide deck every time.
- **Smart scoping.** You can't build everything in 2 days. Show us you know what to prioritize, what to stub out, and what to skip entirely. That judgment is a skill, not a compromise.
- **Honest communication.** If something doesn't work, say so and explain why. If AI tools helped, say how. If you hit a wall, tell us what you tried. We respect honesty over polish.
- **Multitasking & resourcefulness.** You'll need to research an unfamiliar domain, architect a solution, write code, and prepare a presentation — all in 48 hours. That's intentional.
- **Loyalty to the outcome, not the tech.** We don't care if you use Python or Rust. We care that you chose the right tool for the job and can explain why.

Rules & Logistic

- **Timeline:** 48 hours from the time you receive this document.
- **Choose one case study** (Case A or Case B). Attempting both is not expected or rewarded.
- **Solo work only.** You may use any AI tools, libraries, frameworks, or open-source code — but the thinking and integration must be yours.
- **Submit:** GitHub repo (or zip) containing: source code, PRD document, and presentation slides.
- **Demo:** Be prepared to run your demo live during the presentation. Screen recordings are acceptable as backup, but live demo is strongly preferred.
- **Questions:** If you have clarifying questions about the problem domain, use AI tools to research — that's part of the test. For logistics questions only, reach out to the provided contact.